

Python을 사용해 분자를 컴퓨터에서 표현하기

2026년 2학기 약학실험2 · 2주차 · W2

컴퓨터가 어떻게 분자를 표현하는지 이해한다.

SMILES가 Mol object를 거쳐 molecular fingerprint로 변환되는 과정을 확인한다.

01 환경 구성

conda environment · interpreter · jupyter notebook · package

02 분자의 표현법

SMILES · atom · bond · Mol object · molecular graph

03 실제 분자의 정보 추출

Mol object to fingerprint

오늘의 실습 진행 순서

Slides로 개념 확인 → ipynb에서 함께 실습 → Slides로 원리와 과제 정리

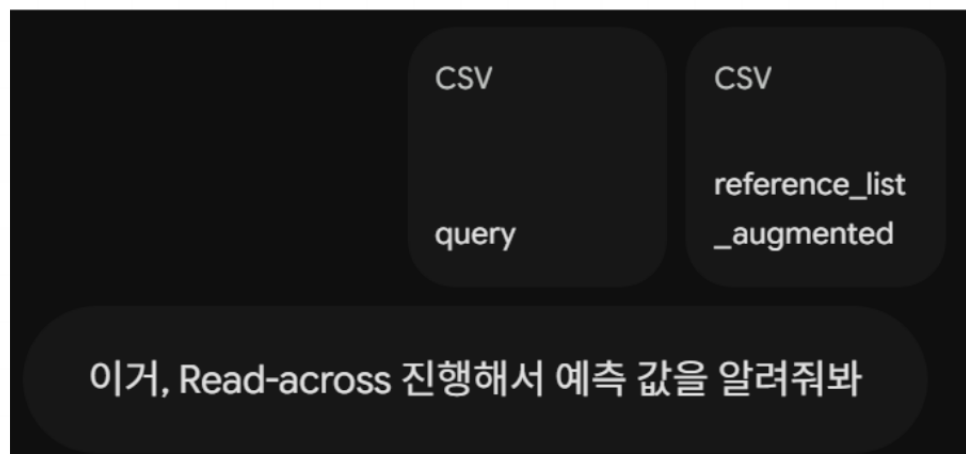
- 01** **지난주 과제 복기** LLM의 Tool calling · Read-across 과정 · 오류가 포함된 응답 확인
- 02** **Python 환경 직접 구성** Miniconda · med-exp 환경 · VS Code · RDKit package 설치
- 03** **Jupyter Notebook에서 실습** 눈으로 보고, 직접 판단해보자
- 04** **분자 표현을 직접 계산해보기**
- 05** **Fingerprint의 원리와 한계** 모든 LLM이 애용한 Morgan은 만능일까?
- 06** **과제와 레포트 안내**

지난 주 과제 복기하기

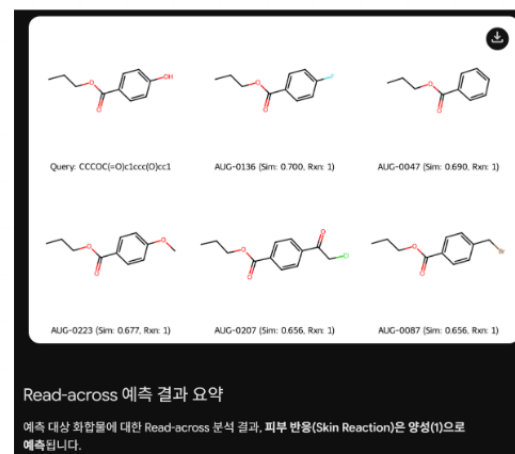
LLM model을 사용하여 Read-across 진행

지난 주 과제

지난 주, 우리는 아무런 배경지식 없이 Read-across를 수행했다. 대체 LLM은 어떻게 Read-across를 수행했을까?



▲ Query 입력 화면



▲ 결과화면

이렇게 만들어진 결과는 AI가 “만들어낸” 결과일까 아니면 실제 프로그램을 돌린 뒤 뽑아낸 것일까?

지난 주 과제 복기하기

LLM model을 사용하여 Read-across 진행

지난 주 과제

ChatGPT든, Gemini든, Claude든, 어딘가에는 “코드 표시” 버튼이 있을 것이다.

```
Python

import pandas as pd

# Load the datasets
try:
    df_query = pd.read_csv('query.csv')
    df_ref = pd.read_csv('reference_list_augmented.csv')

    # Get basic info
    query_info = {
        'columns': df_query.columns.tolist(),
        'shape': df_query.shape,
        'head': df_query.head().to_dict('records')
    }

    ref_info = {
        'columns': df_ref.columns.tolist(),
        'shape': df_ref.shape,
        'head': df_ref.head().to_dict('records')
    }

    print('Query Info:')
    print(query_info)
    print('Ref Info:')
    print(ref_info)
except Exception as e:
    print(f'Error: {e}')

코드 출력
```

▲ Pandas 스크립트(데이터셋 확인)

```
Python
import pandas as pd
from rdkit.Chem import Chem
from rdkit.Chem.rdchem.rdchem import MolFromSmarts
from rdkit.Chem.rdchem.rdchem import MolFromSmarts

def query = pd.read_csv('query.csv')
def ref = pd.read_csv('reference_list_augmented.csv')

query_smiles = query['SMILES'].tolist()
query_mol = Chem.MolFromSmarts(query_smiles)
query_fp = AllChem.GetMorganFingerprintAsNumpy(query_mol, 2, radius=3)

results = []
for idx, row in ref.iterrows():
    ref_smiles = row['SMILES']
    ref_mol = Chem.MolFromSmarts(ref_smiles)
    if ref_mol is not None:
        ref_fp = AllChem.GetMorganFingerprintAsNumpy(ref_mol, 2, radius=3)
        sim = Chem.rdchem.TanimotoSimilarity(query_fp, ref_fp)
        results.append({
            'reference_id': row['reference_id'],
            'query_smiles': query_smiles,
            'similarity': row['similarity'],
            'similarity': sim
        })

df_sim = pd.DataFrame(results)
df_sim = df_sim.sort_values(by='similarity', ascending=False)

top_k = df_sim.head(5)
print("Top 5 Neighbors:")
print(top_k.to_string())

# Predict (e.g., k=5 approach with top 5 or based on threshold)
top_5_reactions = top_k['ik1_reaction'].value_counts()
print("Top 5 Neighbors:")
print(top_5_reactions)
```

▲ 상위 5개 ECFP4 기반 유사물질 탐색

```
Python
```

```
from rdkit.Chem import Draw
from rdkit.Chem.Draw import rdMolDraw2D
import matplotlib.pyplot as plt

mols = [query_mol] + [Chem.MolFromSmiles(smiles) for smiles in top_k['SMILES']]
legends = ["Query: CCCOC(=O)c1ccc(O)cc1"] + [f"row['reference_id']" (Sim: row['similarity']) for row in top_k['similarity']]

img = Draw.MolsToGridImage(mols, molsPerRow=3, subImgSize=(300, 300), legends=legends)
img.save('read_across_results.png')
print("Image saved as read_across_results.png")
```

코드 출력

```
Image saved as read_across_results.png
```

▲ 이미지 출력

실제로는 python 코드로 스크립트를 작성한 뒤 실제로 컴퓨터 자원을 사용하여 "계산"한 뒤 그 결과를 출력한 것에 가깝다. 이것이 "Tool calling"이다. LLM이 "계산"을 직접 수행한 것이 아니라, **계산을 수행할 수 있는 프로그램을 호출**한 것이다.

근데 우리가 왜 배워야 하는 거죠?

우리가 무슨 결과를 뽑은 건지는 알아야 하지 않겠나...

LLM이 프로그램을 호출한다면, 우리는 그 프로그램이 무엇을 계산하는지 판단할 수 있어야 한다.

Gemini11명 Gemini 3.1 Pro2명 3.6 사고 모델1명 Gemini Pro2명 Gemini · 세부 모델 미기재6명	Claude4명 Opus 5 (심지어 Clude code)1명 Sonnet 51명 세부 모델 미기재2명	ChatGPT / GPT4명 GPT 6.0 Pro1명 세부 모델 미기재3명	기타 · 미기재2명 GitHub만 기재 · 모델 불명1명 AI 모델·서비스 미기재1명
---	--	--	--

어떤 방식으로 계산했을까?

N=21 빅데이터 통계

같은 문제도 분자 표현과 비교 기준에 따라 결과가 달라진다. 어떤 방법을 사용했는지 확인해야 한다.

01

프로그래밍 언어

미기재도 사실 다 뜯어보면 Python 기반입니다.
무조건 Python 배워야겠죠?

Python	2개
언어 미기재	19개

02

분자 구조를 확인한 방법

Fingerprint · MCS

Morgan · ECFP 계열	16개
MACCS	6개
RDKit path · Topological	4개
MCS · 최대 공통 부분구조	2개
미확인	5개

03

유사도 지표

계산 수식은 뭘 썼을까?

Tanimoto	17개
Levenshtein · 문자열 비교	1개
Euclidean 거리 · 물성 비교	1개
미확인	4개

우리가 왜 배운다고 했죠?

LLM이 사용한 방식은
이 분자들을 비교하기에 적절했을까?

01 익숙한 방법에 집중되어 있다.

Tanimoto가 17개 레포트에 등장한다. 여러 방법을 사용한 경우도 있지만, Molecular Fingerprint는 Morgan·ECFP, MACCS, RDKit FP만이 사용되었다.

02 여러 Fingerprint를 사용했지만 정작 해당 방법이 맞는지 안 봤다.

MACCS를 사용하면 유사도가 100%가 나오는 물질이 있었음에도 LLM은 이상하다 판단하지 않았다.

03 널리 쓰인다는 것이 정답이라는 뜻은 아니다.

훨씬 크고 다양한 화학물질 집합에서도 같은 기준이 통할까? LLM의 설명이 익숙한 예제 수준에 머물지는 않았는지 확인해야 한다.

이제부터 직접 코드를 실행하고, 표현과 지표를 바꾸며 결과를 확인해보자!

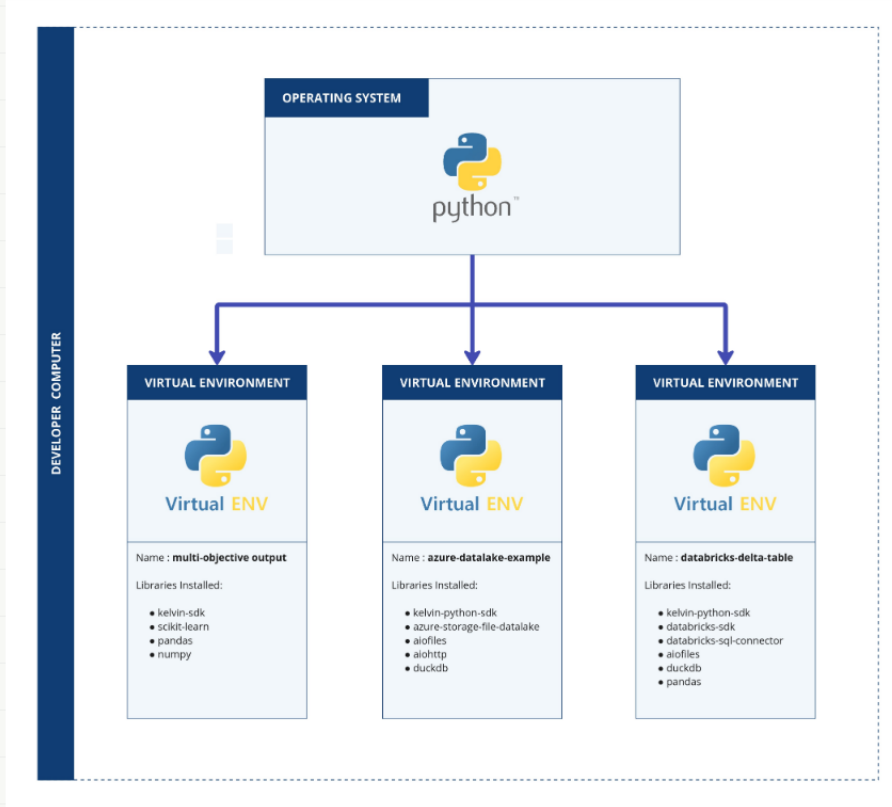
환경 설치

이제 컴퓨터를 준비해주세요

스크립트를 읽는 방법에 대해서 배울 예정입니다!

환경 설치

가상 환경 만들어보기



▲ 가상 환경 개요

가상 환경이란?

한 컴퓨터 안에서 필요한 **Python 버전**과 **package 조합**을 분리해 두는 공간이다. 쉽게 생각해서 **컴퓨터 안에 여러 버전의 동일한 프로그램이 설치될 수 있게 만드는 것이다.**

HWP로 쓰인 문서를 읽으려니, 너무 **구버전이어서 한글 2002**만을 요구하고 어디서는 HWPX로 쓰여 있어서 **한글 2020 이상**만 읽을 수 있다면,

가상 환경은 이를 2개로 분리하여 한글 2002가 설치된 프로필과 한글 2020이 설치된 프로필을 만들어서 서로 영향을 주지 않고 사용할 수 있다.

멀쩡히 쓰던 코드 명령어가 갑자기 바뀌면서 특정 버전만을 요구하는 경우가 많아서 이게 정말 중요하다!

환경 설치

가상 환경 만들어보기

직접 조작

우리는 가상 환경을 만들기 위해 **Conda** 계열 툴을 설치한다.
실습에는 **Miniconda**를 사용한다.

01 다운로드 페이지 열기
[Anaconda.com/download](https://anaconda.com/download) 접속

02 회원가입 스킵
화면 우측에서 **Skip Registration** 클릭

Get Started with Anaconda Distribution

Install Python, Jupyter, and thousands of data science packages in one step. Trusted by over 50 million users who need tools that work—without the setup headaches.

What's included in Anaconda Distribution? ▾

- ✓ Thousands of vetted Python packages with dependency management
- ✓ Conda package and environment manager
- ✓ Jupyter Notebook and JupyterLab
- ✓ Desktop app (Navigator) or command line interface
- ✓ Works on Windows, macOS, and Linux

[See documentation](#) for more details and step-by-step instructions.

What's included in my free Anaconda account? ▾

Added Benefits for Academic Institutions ▾



Download Now

Get access in 30 seconds.*

Get Started >

Returning Users >

*Subject to our [Terms of Service](#). Use of Anaconda's offerings at an organization of more than 200 employees/contractors requires a paid business license unless your organization is eligible for discounted or free use. [See Discard](#)

Skip Registration >



▲ Anaconda Download 화면에서 오른쪽 상단의 Skip Registration을 클릭한다.

환경 설치

가상 환경 만들어보기

Choose Your Download

Windows

Mac

Linux

Anaconda Distribution

Package and environment management with conda, Anaconda Navigator desktop app, and 600+ packages. Everything you need for data science.

GRAPHICAL INSTALLER

[↓ Windows 64-Bit Graphical Installer](#)

[📘 Graphical install instructions](#)

COMMAND LINE

[📖 CLI install instructions](#)

Miniconda

Lightweight installer that only includes conda, Python, and their dependencies. Install only what you need.

GRAPHICAL INSTALLER

[↓ Windows 64-Bit Graphical Installer](#)

[📘 Graphical install instructions](#)

COMMAND LINE

[📖 CLI install instructions](#)

Need help choosing an installer? [Compare Anaconda Distribution vs. Miniconda →](#)

▲ Miniconda 설치 화면

환경 설치

가상 환경 만들어보기

1. Next를 조심스럽게 누르다가 **Just Me (recommended)** 선택
2. 설치 경로는 기본값 그대로 두고 Next
3. **Create shortcuts** 체크 확인 후 Install

완료되면 **시작** 메뉴에서 **Anaconda Prompt**를 찾아 실행한다.

```
(base) C:\Users\사용자>
```

위처럼 **(base)**가 붙은 상태로 프롬프트가 뜨면 **성공**. 아래를 입력해보자.

```
(base) C:\Users\사용자> conda create -n <이름(영어만 가능)> python=3.12  
(base) C:\Users\사용자> conda create -n med-exp python=3.12
```

```
Do you accept the Rerms of Service (ToS) [(a)ccept/(r)eject/(v)iew]: a
```

3개의 동의서가 나오는데 모두 **a**를 입력해 동의한다. 그리고 마지막에 **Proceed ([y]/n)?**가 나오면 **y**를 입력한다.

환경 설치

가상 환경 만들어보기

```
(base) C:\Users\사용자> python --version
```

<뭐라고 나올까>

```
(base) C:\Users\사용자> conda activate med-exp
```

```
(med-exp) C:\Users\사용자> python --version
```

```
Python 3.12.14
```

환경 설치

가상 환경 만들어보기

```
(base) C:\Users\사용자> python --version  
Python 3.14.7
```

```
(base) C:\Users\사용자> conda activate med-exp  
(med-exp) C:\Users\사용자> python --version  
Python 3.12.14
```

```
(med-exp) C:\Users\사용자> python
```

환경 설치

가상 환경 만들어보기

```
(base) C:\Users\사용자> python --version
```

```
Python 3.14.7
```

```
(base) C:\Users\사용자> conda activate med-exp
```

```
(med-exp) C:\Users\사용자> python --version
```

```
Python 3.12.14
```

```
(med-exp) C:\Users\사용자> python
```

```
Python 3.12.14 | packaged by Anaconda, Inc. | (main, 날짜) [MSC v.1942 64 bit (AMD64)] on win32
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>>
```

환경 설치

가상 환경 만들어보기

```
(base) C:\Users\사용자> python --version
```

```
Python 3.14.7
```

```
(base) C:\Users\사용자> conda activate med-exp
```

```
(med-exp) C:\Users\사용자> python --version
```

```
Python 3.12.14
```

```
(med-exp) C:\Users\사용자> python
```

```
Python 3.12.14 | packaged by Anaconda, Inc. | (main, 날짜) [MSC v.1942 64 bit (AMD64)] on win32
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> print("Hello, World!")
```

환경 설치

가상 환경 만들어보기

```
(base) C:\Users\사용자> python --version
```

```
Python 3.14.7
```

```
(base) C:\Users\사용자> conda activate med-exp
```

```
(med-exp) C:\Users\사용자> python --version
```

```
Python 3.12.14
```

```
(med-exp) C:\Users\사용자> python
```

```
Python 3.12.14 | packaged by Anaconda, Inc. | (main, 날짜) [MSC v.1942 64 bit (AMD64)] on win32
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> print("Hello, World!")
```

```
Hello, World!
```

```
>>>
```

환경 설치

가상 환경 만들어보기

```
(base) C:\Users\사용자> python --version
```

```
Python 3.14.7
```

```
(base) C:\Users\사용자> conda activate med-exp
```

```
(med-exp) C:\Users\사용자> python --version
```

```
Python 3.12.14
```

```
(med-exp) C:\Users\사용자> python
```

```
Python 3.12.14 | packaged by Anaconda, Inc. | (main, 날짜) [MSC v.1942 64 bit (AMD64)] on win32
```

```
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> print("Hello, World!")
```

```
Hello, World!
```

```
>>>
```

이 상태에서 python 스크립트를 작성할 수는 없잖아요...

환경 설치

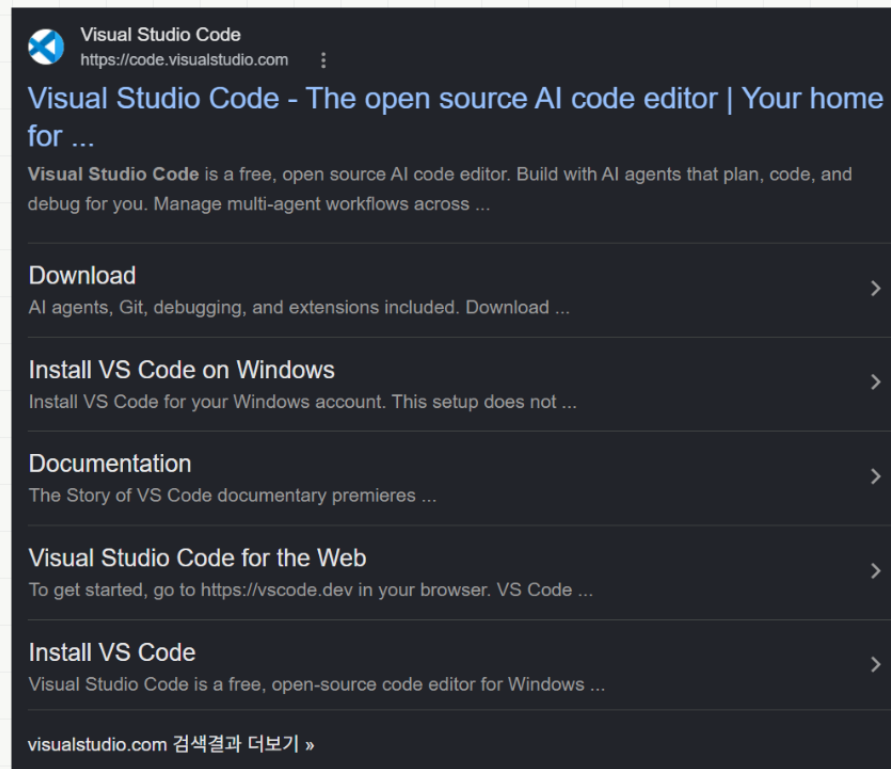
코드 편집기 설치

그래서 필요합니다. **VS Code!**

HWP, Word, 메모장, LibreOffice, Notepad++ 처럼 문서를 편집하는 프로그램이 있듯이, Python 코드를 편집하고 작성하는데 도움을 주는 편집 프로그램이 있다. 그 중에서 가장 보편적으로 쓰이는 것이 **VS Code**이다.

* 참고로 [WindSurf](#), [Cursor](#) 같은 AI 기반 코드 편집기들도 있다. 현재는 **VS Code**도 충분히 AI 기반으로 전환되었다.

인터넷 창에 **VS Code**를 검색해서 다운 받아보자!



▲ Visual Studio 경량판이라고 하면 틀린 설명이지만 이해는 쉽다.

환경 설치

코드 편집기 설치

1. 동의합니다.
2. 설치 경로는 기본으로 두고 다음
3. 시작 메뉴도 기본으로 두고 다음
4. 이상한 체크들도 그냥 두고 다음
5. 설치
6. 실행 후 오른쪽 하단에 `Continue without signing in` 클릭
7. 마음에 드는 테마를 선택한 뒤에 `Get Started` 클릭

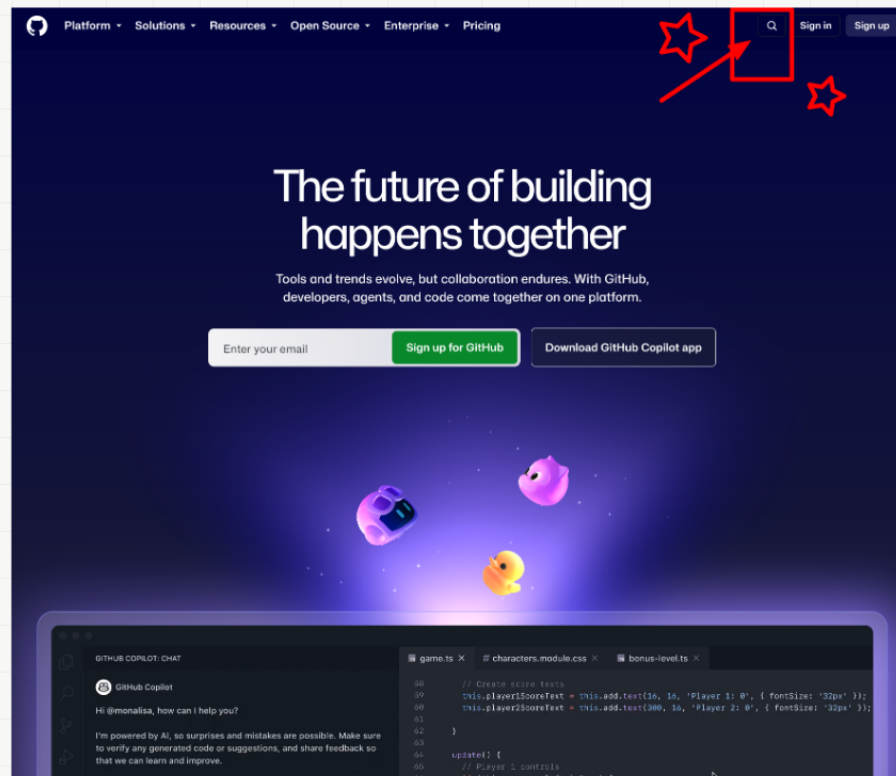
여기까지 수업이 20분 소요 되었겠죠?

수업 자료 받으러가요

Github에서 수업 자료 받자

Github.com 으로 접속하자. 오른쪽 상단에서

HY-TOX를 검색하자.

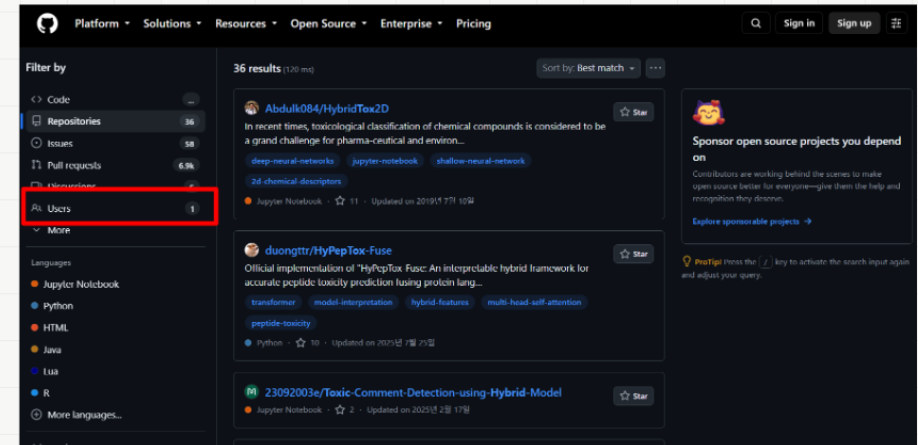


▲ Github 메인 화면

수업 자료 받으러가요

Github에서 수업 자료 받자

Users를 누르고, HY-TOX-2026 유저를 클릭한다.



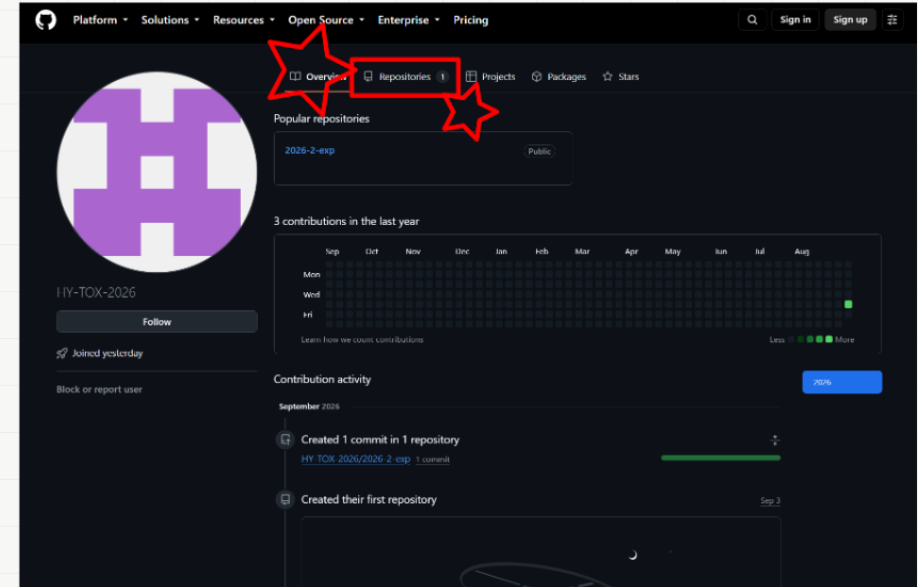
▲ 검색 후 화면

수업 자료 받으러가요

Github에서 수업 자료 받자

Repositories를 클릭하고 2026-2-exp를 클릭한다.

아니면 아래에 있는 Popular repositories에서 2026-2-exp를 클릭해도 된다.



▲ Repositories 클릭!

준비 — Anaconda Prompt 열기

Notebook 실습 준비

혹시나 방금 설치한 Anaconda Prompt를 닫았다면,

Windows **시작 메뉴**에서 Anaconda Prompt를 검색하여 실행한다.

Anaconda Prompt가 열리면 방금 전에 만든 med-exp 환경을 활성화한다.

```
(base) C:\Users\사용자> conda activate med-exp  
(med-exp) C:\Users\사용자>
```

명령어 앞의 환경 이름이 (base)에서 (med-exp)로 바뀌었는지 확인한다.

준비 — 필요한 package 설치

Notebook 실습 준비

(med-exp)가 표시된 **Anaconda Prompt**에서 다음 명령어를 입력한다.

```
(med-exp) C:\Users\사용자> conda install -c conda-forge rdkit pandas ipykernel
```

명령어	의미
conda install	현재 활성화된 환경에 package를 설치한다.
-c conda-forge	package를 받을 channel로 conda-forge를 지정한다.
rdkit pandas ipykernel	설치할 package 목록이다.

설치 여부를 묻는 Proceed ([y]/n)?가 나타나면 y를 입력하고 를 누른다.

설치가 끝나면 Notebook(VS Code에서 ipynb 파일을 열면 된다.)을 열고 오른쪽 위에서 Python (med-exp) kernel을 선택한다.

Library가 뭐예요. 프로그램이란 같은 말 아닌가?

Library란?

Library는 다른 코드에서 가져다 쓸 수 있도록 기능을 모아둔 코드 묶음이다.

Program은 특정 작업을 수행하도록 작성한 코드이며, 필요한 Library를 활용할 수 있다.

둘은 역할로 구분한다. RDKit은 Library이고, 이를 이용해 분자 유사도를 계산하는 Python 스크립트는 Program이다.

```
1 RDKit library
2   └─ rdkit package
3       └─ Chem                               분자를 읽고 다룰 때 쓰는 기능들의 모음집
4           └─ MolFromSmiles()               function (함수)
5           └─ Mol                           class
6       └─ Chem.Draw                         분자 구조 그리는 기능들의 모음집
7           └─ MolsToGridImage()             function (함수)
8       └─ DataStructs                       bit vector 비교할 때 쓰는 모듈 (큰 단위)
9           └─ TanimotoSimilarity()          function (함수)
```

- **Package/module**은 기능을 주제별로 정리해 둔 코드 묶음이다.
- **Function(함수)**은 입력을 받아 정해진 작업을 하고 결과를 돌려주는 기능이다.

$y = ax + b$ 처럼 수학에서 $f(x)$ 를 정의하듯이, Python에서는 `def 함수이름(변수 입력)`로 함수를 정의한다.

Function은 뭔데요

Function에 대한 이해

수학으로 표현하면

01 · 입력이 하나일 때

$$f(x) = ax + b$$

a, b 는 미리 정한 값이고, x 를 입력받는다.

02 · 입력이 세 개일 때

$$f(x, a, b) = ax + b$$

x 뿐 아니라 a, b 도 함께 입력받는다.

Python으로 표현하면

01 · x 를 입력받는다.

```
1 # a, b는 미리 정해두어야 한다.  
2 def 함수이름(x):  
3     y = a*x + b  
4     return y
```

02 · x, a, b 를 입력받는다.

```
1 def 함수이름(x, a, b):  
2     y = a*x + b  
3     return y
```

함수를 호출하면, 입력값이 순서대로 전달된다.

$$f(2, 3, 4) = 3 \times 2 + 4 = 10$$

$x = 2, a = 3, b = 4$ 로 계산하며, `return y`는 계산한 값을 돌려준다.

함수이름(2, 3, 4) → 10

설치한 기능은 `import`해서 사용한다

Python 기본 문법

```
1 from rdkit import Chem
2 from rdkit.Chem import Draw
3
4 mol = Chem.MolFromSmiles("CCO")
5 image = Draw.MolToImage(mol)
```

코드	가져온 것	사용하는 기능
<code>from rdkit import Chem</code>	분자 처리 module/package	<code>Chem.MolFromSmiles()</code>
<code>from rdkit.Chem import Draw</code>	구조 그리기 module	<code>Draw.MolToImage()</code>

설치는 컴퓨터 어딘가에 함수를 보관해 놓는 것이고, `import`는 어딘가에 있던 친구를 로딩해놓는거다.

마치 요리 레시피를 사서 집에 보관하는게 설치라면, 요리할 때 레시피를 꺼내는게 `import`이다.

저녁 식사 라는 작업을 수행하고 싶으면 요리책에서 A 음식과 B 음식을 하계끔 작동 요령을 적는게 스크립트다.

어떤 module과 function이 있는지는 공식 문서에서 찾는다

RDKit · 코딩 관련 문서는 다 인터넷에 있다

RDKit에는 기능이 매우 많아서 다 외울 수도 없다. 이것을 전부 외우는 대신 공식 문서에서 목적에 맞는 **module**과 **function**을 찾는다.

하고 싶은 일	뭘 찾아야하죠	찾은 함수
SMILES를 분자로 바꾸기	<code>rdkit.Chem</code>	<code>MolFromSmiles()</code>
분자 구조 그리기	<code>rdkit.Chem.Draw</code>	<code>MolToImage()</code> , <code>MolsToGridImage()</code>
분자 특성 계산하기	<code>rdkit.Chem.Descriptors</code>	<code>MolWt()</code> , <code>MolLogP()</code>
fingerprint 비교하기	<code>rdkit.DataStructs</code>	<code>TanimotoSimilarity()</code>

1. [RDKit Documentation](#)에서 **Getting Started with the RDKit in Python**을 보면 된다.
2. 더 자세한 목록은 [Python API Reference](#) 또는 [Python Module Index](#)에서 찾으면 된다!
3. 이제는 LLM(Chatbot)도 직접 홈페이지를 조회하여 관련 기능을 검색하기 때문에 특정 기능의 모듈, 함수가 있는지, 어떻게 쓰는지까지 Chatbot이 알려준다. 하지만 **공식 문서에서 직접 확인**하는 습관을 들이면 완전 신규 모듈, 함수가 나왔을 때 빠르게 습득할 수 있다.

from pathlib import Path 읽기

Import는 알겠는데 왜 from ~ import를?

```
1 from pathlib import Path
```

코드	의미
pathlib	파일과 폴더 경로를 다루는 Python 표준 library 의 module
Path	pathlib module 안에 정의된 class
from ... import ...	module 안에 있는 특정 기능을 직접 가져온다.

```
1 DATA_DIR = Path("data")      # pathlib.Path("data")랑 같은데 매번 이렇게 길게 쓰기 귀찮으니!  
2 Query = pd.read_csv(DATA_DIR / "query.csv")
```

현재 Notebook이 실행되고 있는 경로를 기준으로 "data" 라는 폴더를 찾는게 Path("data")이다.
/ "query.csv" DATA_DIR 변수에 있는 현재 경로 + data 에서 또 /"query.csv"를 붙여서 본다는 이야기이다.
pd.read_csv()는 csv 파일을 읽어서 DataFrame으로 만들어준다는 것이다.

HANDS-ON PRACTICE

실습을 진행하자.

다운로드했던 ipynb 파일을 열어보자!

Fingerprint마다 분자를 보는 방법이 다르다

Fingerprint의 원리 · 같은 분자를 서로 다른 방식으로 본다

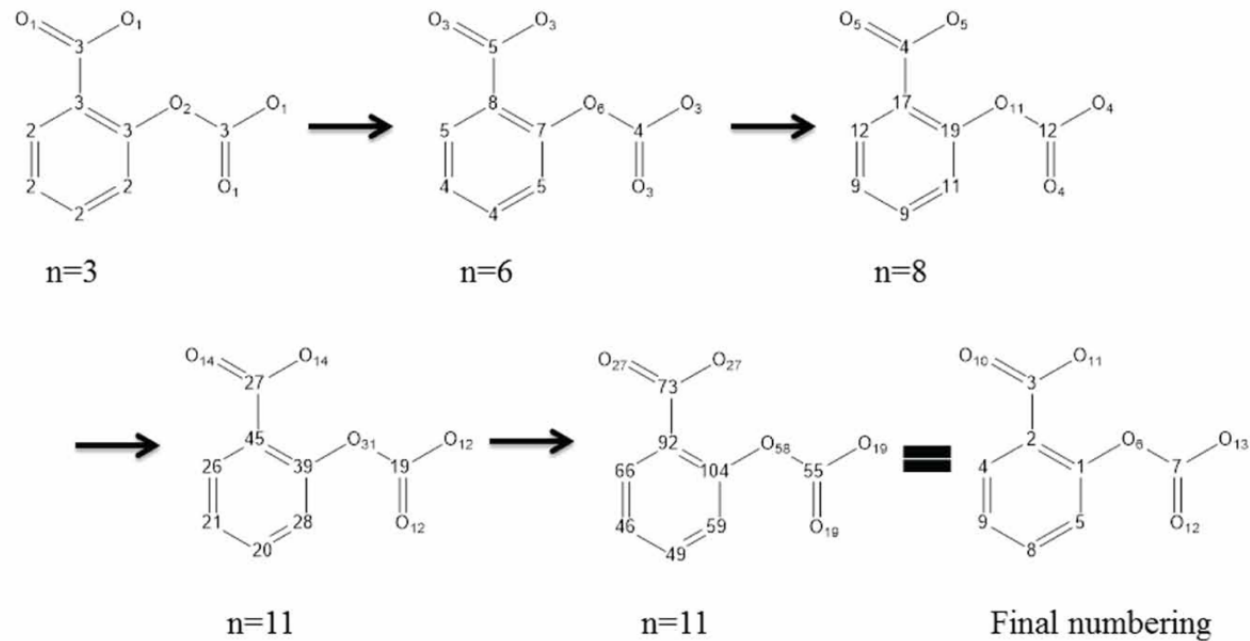
Fingerprint	애들이 보는 방식	특징
Morgan / ECFP4	각 원자 주변 환경은 어떻게 구성되어있는가?	원형으로 주변을 확장
Atom Pair	어떤 원자 두 개가 얼마나 떨어져 있는가?	원자 type과 최단거리
MACCS keys	미리 정한 구조 key가 존재하는가?	166개의 key
PubChem	원소 수·고리·원자 환경·부분구조가 존재하는가?	881개의 공개 key

Fingerprint는 분자 구조에 대해서 **각자의 규칙으로 관찰한 특징**을 숫자로 바꾼 것이다. 그 규칙은 수 조개가 넘는 화학 공간을 모두 표현할 수는 없다!

ECFP는 Morgan algorithm을 기반으로 한다. Morgan algorithm이란?

Morgan algorithm

Figure 2. Application of the Morgan algorithm to iteratively constructs the atomic connectivity values. The final numbering of atoms 1, 2, 3, ..., etc., follow the descending values 104, 92, 73, 66, etc., as is indicated in the figure outside the sequence box.



▲ 이웃 원자의 connectivity 값을 반복해서 더하여 각 원자의 identifier를 갱신하는 Morgan algorithm. 출처: Joseph et al. (2019), Figure 2.

각 반복에서 이웃 원자의 정보를 합쳐 identifier를 갱신한다. **ECFP**는 이 원리를 화학적 atom invariant와 함께 사용하는 circular fingerprint 계열이다.

ECFP4의 4는 대체 무슨 의미인가?

Morgan radius 2 ≈ ECFP4 · 중심 원자에서 bond 두 칸까지

	반복	관찰 범위	propyl chain에서 보는 예
	radius 0	중심 원자 자체	이 원자는 탄소인가? 산소인가?
	radius 1	중심 + 한 bond 이웃	이 탄소는 탄소와 산소에 연결됐는가?
	radius 2	중심 + 두 bond 이웃	두 칸 안에 carbonyl이나 aromatic carbon이 있는가?



```
1 morgan_generator = rdFingerprintGenerator.GetMorganGenerator(  
2     radius=2,  
3     fpSize=2048,  
4 )
```

ECFP4는 중심에서 radius 2까지 보므로, 관찰하는 환경의 diameter가 최대 4 bond가 된다.

Morgan / ECFP4가 놓칠 수 있는 것

긴 반복 구조에서는 정보를 구분하지 못하게 된다.

```
1 Phenyl-C6  
2 Phenyl-C9  
3 Phenyl-C14  
4 Phenyl-C25  
5 Phenyl-C30
```

radius 2 · 2048 bit → Query와 Tanimoto 1.000

- 충분히 긴 사슬에서는 radius 2 안의 **환경**이 계속해서 반복된다.
- fpSize를 2048에서 4096로 늘려도 이 사례는 해결되지 않는다!
- radius를 4로 늘리면 짧은 사슬은 다시 구분할 수 있게 되지만, 다시 또 긴 사슬에서는 다시 구분하지 못 한다.

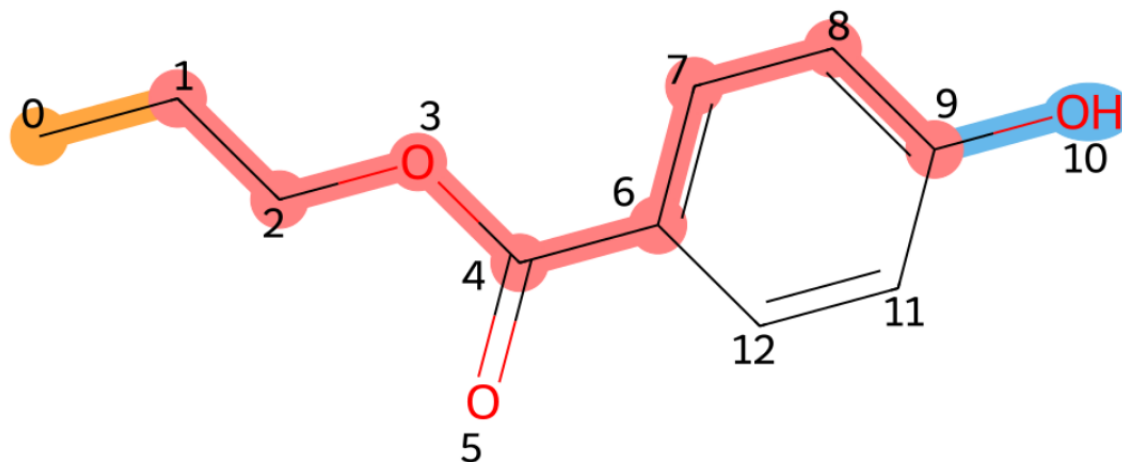
따라서 Morgan fingerprint (ECFP4)로 분자 구조가 상당히 유사하게 나왔다고 해서, 그 수치를 곧이곧대로 믿을 순 없다.
이것은 단순히 ECFP4의 관점으로 봤을 때 유사하다는 것이며, 정말 **유사**하다는 것인지는 약학 도메인으로 확인해야 하는 것이다!

Atom Pair는 원자 두 개와 그 사이 거리를 이용하여 분자를 해석한다

Atom Pair fingerprint · 분자 전체에서의 거리 관계

Atom Pair = atom type A + topological distance + atom type B

Highlighted bonds show the shortest path in the molecular graph (not a 3D distance).



Atom 0 -- shortest path: 9 bonds -- Atom 10

▲ Propyl paraben에서 선택한 두 원자와 그 사이의 최단 bond 경로. RDKit으로 작성.

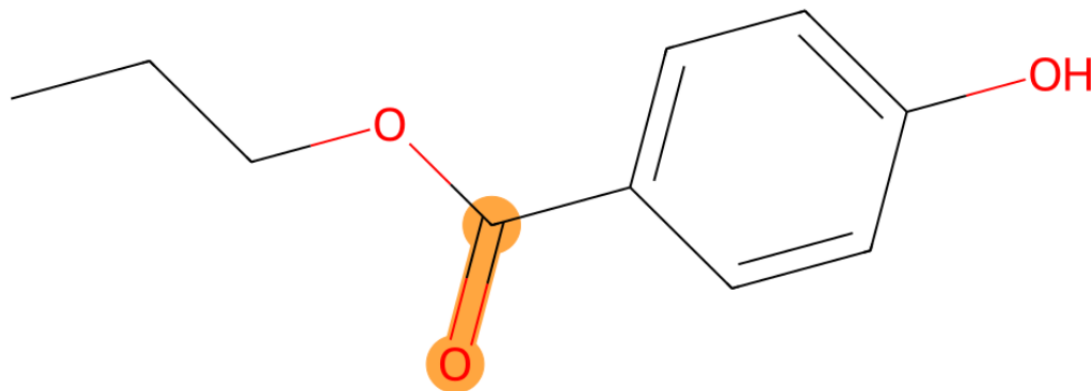
Atom Pair = 원자 A의 type + 두 원자 사이의 최단 bond 수 + 원자 B의 type. 같은 부분구조가 있어도 원자의 배치와 거리가 달라지면 서로 다른 pair가 만들어진다.

MACCS는 화학자가 중요하다고 생각한 166가지의 구조 패턴을 체크리스트로 반환하는 것이라 보면 된다

MACCS structural keys · 각 구조에 대해 Yes or No

MACCS asks a fixed list of structural questions

A matching SMARTS pattern switches its predefined key ON.



MACCS key 154 | [#6]=[#8] | carbonyl C=O present -> 1

▲ Propyl paraben에서 MACCS key 154의 carbonyl pattern이 일치하는 위치

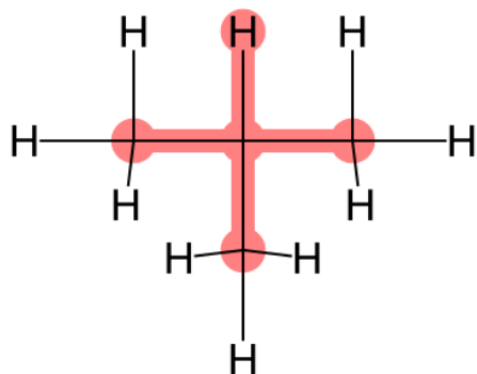
- MACCS는 hash로 위치를 만드는 대신 **미리 정의된 166개의 구조**를 차례로 검사한다.
- 예시의 key 154는 `[#6]=[#8]`, 즉 carbonyl `C=O`가 존재하는지를 체크한 것이다.
- 재밌게도 RDKit에서 제공하는 MACCS는 key 번호를 1부터 맞추기 위해 167-bit vector를 반환한다.

PubChem fingerprint는 881개의 key를 사용한다 마치 MACCS처럼!

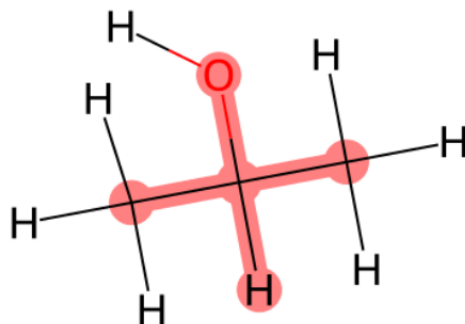
PubChem substructure fingerprint · 훨씬 다양한 패턴의 구조

PubChem keys can be traced back to published structural definitions

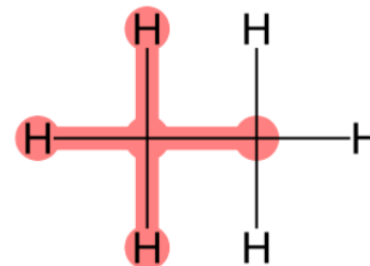
Orange atoms show the center atom and the neighbors tested by each example key.



bit 335 | C(C)(C)(C)(H)



bit 339 | C(C)(C)(O)(H)



bit 374 | C(H)(H)(H)

▲ PubChem fingerprint의 atom-neighborhood key 세 가지 예시. 주황색은 조건을 만족하는 중심 원자와 이웃 원자이다.

Element count, ring, simple atom pair, atom neighborhood, detailed substructure처럼 서로 다른 범주의 패턴들을 합쳐 **881-bit vector**를 만든다.

그래서 PubChem bit는 번호의 정의를 찾아서 Query와 reference가 왜 그 위치에서 달라졌는지 확인할 수 있다.

Morgan은 특정 bit가 어떤 구조를 의미하는지 알 수 없지만 MACCS나 PubChem은 bit 정보로부터 분자 구조 해석이 가능하다.

어떤 fingerprint를 써야 할까?

목적에 따라 선택해야한다! · 하나의 fingerprint가 모든 구조 관계를 설명하지 않는다

목적	먼저 고려할 fingerprint	함께 확인할 점
국소적인 구조 환경 비교	Morgan / ECFP4	radius, bit/count, fpSize
원자 사이의 거리 관계 비교	Atom Pair	atom type, distance, count simulation
정해진 구조 key의 해석	MACCS	구현별 key 차이
다양한 공개 구조 key 확인	PubChem	881-bit 정의와 구현 차이

- 1. 분석 목적에 맞는 fingerprint를 선택한다.
- 2. 같은 Query를 둘 이상의 fingerprint로 비교한다.
- 3. similarity 숫자만 보지 말고 실제 구조와 차이 bit를 함께 확인한다.

Fingerprint는 정답을 알려주는게 아니라, 분자를 바라보기 위해 선택하는 **관찰 기준**이다.
그리고 그 관찰 기준은 연구자, 사용자가 직접 선택해야만 한다! 유사도에 확립된 기준이라는 것은 없기 때문에!

분자를 해석하는 방법은 이것만 있는게 아니다.

문자열에서 2D molecular graph, 3D conformer까지 · 사용하는 정보가 달라진다

관점	무엇을 입력으로 보는가?	대표적인 fingerprint
문자열 기반	SMILES에 나타난 문자와 token의 배열	LINGO, MHFP, SECFP
2D · local environment	각 원자 주변의 연결 환경	ECFP/Morgan, FCFP, EState
2D · path / pair	bond path 또는 두 원자 사이의 topological distance	RDKit FP, Atom Pair, Topological Torsion
2D · structural key	미리 정의된 원소·고리·부분구조	MACCS, PubChem, Klekota–Roth, Pattern FP
2D · hybrid	circular substructure와 atom-pair distance를 함께 사용	MAP4
3D · shape / geometry	conformer의 좌표, 거리, 원자 특성	E3FP, USR/USRCAT, ElectroShape, GETAWAY, WHIM, MoRSE, RDF

2D는 molecular graph의 연결 관계를 보고, 3D는 실제로 생성하거나 측정한 conformer 좌표까지 사용한다.
따라서 3D 결과는 conformer 생성 방식과 protonation·stereochemistry에도 영향을 받는다.

그리고 누군가가 이것들을 한데 모아 Package로 만들어줬다.

RDKit과 scikit-fingerprints

RDKit의 주요 generator

- Morgan / Feature Morgan
- RDKit topological FP
- Atom Pair
- Topological Torsion
- Pattern / Layered FP
- MACCS keys

더 많은 2D 표현

- Avalon, EState, ERG
- Autocorrelation, BCUT2D
- Functional Groups
- Klekota–Roth, Laggner
- MQNs, VSA, Ghose–Crippen
- PubChem / CACTVS

3D 좌표를 사용하는 표현

- E3FP
- ElectroShape
- Pharmacophore
- USR / USRCAT
- GETAWAY, WHIM
- MoRSE, RDF

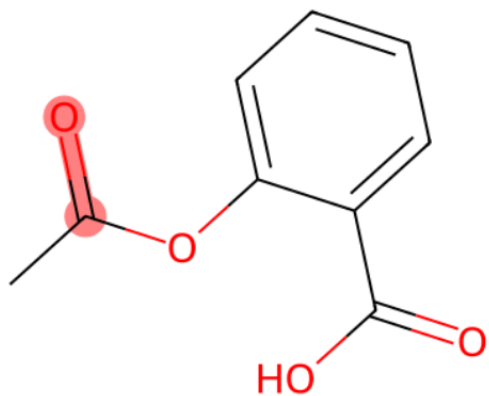
RDKit fingerprinting 문서와 scikit-fingerprints 목록을 보면 구현된 방법과 parameter를 확인할 수 있다.

과제 설명! 예시, MACCS bit를 구조로 되돌려 보기

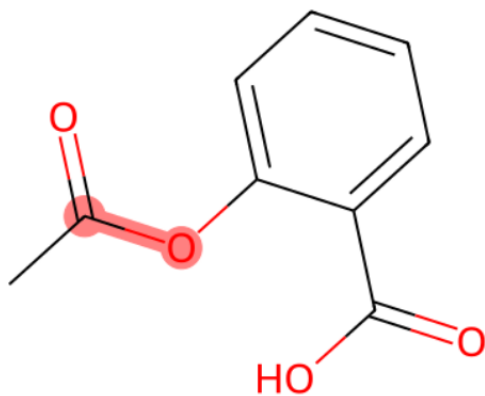
ON bit 번호만 나열하지 말고 · 실제로 일치한 substructure를 확인해보자

Example: connect each MACCS ON bit to the matching substructure

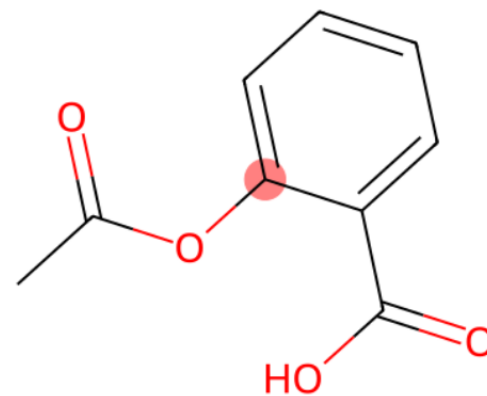
The same aspirin molecule is redrawn; orange marks the match for each predefined key.



ON bit 154 | [#6]=[#8]



ON bit 157 | [#6]-[#8]



ON bit 162 | a

▲ Aspirin에서 ON인 MACCS key 세 개와 각 key에 일치하는 구조 위치. 같은 분자를 반복해서 그리고 key마다 일치 부위를 주황색으로 표시했다.

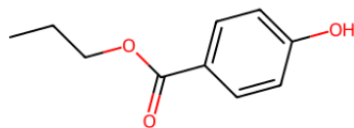
같은 분자라도 어떤 key를 체크하느냐에 따라 강조되는 구조가 달라진다. 과제에서는 이런 식으로 분자간의 서로 다른 차이를 실제 구조 그림으로 확인하고, 그 차이와 유사도 점수가 정말 유사하다고 볼 수 있는지, 본인의 논리로 설명하는 것이 핵심이다.

과제 · 같은 점수, Query와 다른 bit 위치

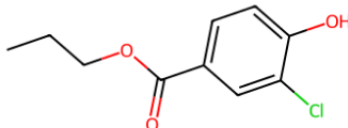
차이 bit 수와 Tanimoto는 같지만 Query와 달라진 loci는 다르다

Three teams: equal Tanimoto, different loci relative to Query

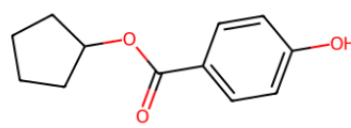
A and B have equal difference counts, but their changed bit positions against Query are not the same.



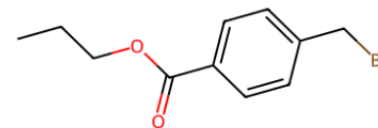
Query



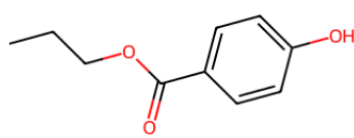
Team 1 A | Query Tanimoto 0.800



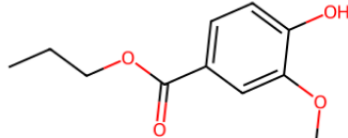
Team 2 A | Query Tanimoto 0.6923



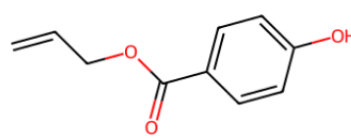
Team 3 A | Query Tanimoto 0.6667



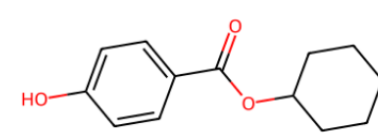
Query



Team 1 B | Query Tanimoto 0.800



Team 2 B | Query Tanimoto 0.6923



Team 3 B | Query Tanimoto 0.6667

▲ 각 team의 A와 B는 Query에 대한 MACCS Tanimoto와 차이 bit 수가 같지만, Query와 비교해 달라진 bit loci는 서로 다르다.

점수와 차이 개수만 보면 A와 B는 Query에 똑같이 유사하다. 그러나 Query와 달라진 bit의 위치와 실제 구조를 보면...?

과제 · MACCS ON bit와 substructure 비교

Query와 reference 비교 · 코드, bit 정의, 구조 그림을 하나의 근거로 연결하기

1. Query와 지정된 reference의 MACCS fingerprint에서 값이 1인 bit를 추출한다.
2. 공통 ON bit와 한쪽에만 존재하는 ON bit를 구분한다.
3. 선택한 차이 bit의 MACCS 정의 또는 SMARTS pattern을 확인한다.
4. 각 분자에서 실제로 일치한 substructure를 **색으로 강조한 이미지**로 출력한다.
5. Query에 대해 A와 B의 이미지를 나란히 놓고, **둘 중 어느 분자가 Query와 더 유사하다고 판단되는지** 나름의 논리로 설명한다.

AI를 사용해 코드 작성과 오류 해결을 진행해도 됩니다. 그건 요새 AI가 잘 해주니까요. 다만! 본인의 생각을 펼쳐볼 수 있어야 합니다!

추가 과제 · 분자를 나타내는 방법, Graph란?

Fingerprint처럼 하나의 **vector**(숫자)로 압축하지 않는 방식

데이터 구조에서 사용하는 Graph의 개념과

화학구조를 나타내는 **molecular graph**의 개념을 조사한다.

자유 형식입니다! 다만, 출처는 확실해야 합니다. (논문, 공식 문서, GitHub, 등) (예외적으로 영문 위키피디아는 받지만, 나무위키는 안 받습니다.)

기본적으로 출처는 “공인된 인물만 쓸 수 있는 곳”, 아무나 못 쓰는 곳이어야 합니다. 위키피디아 인용 시 되도록이면 하단에 있는 “출처”를 꼭 확인해서 위키피디아 대신, 원출처를 써보세요. AI한테 출처 정리하라고 하면 또 서지 정보를 잘 알려줄겁니다.

레포트 설명

재현 · 본인의 논리로 설명 · 원리와 한계 조사 · 출처 기록

실습 코드 재현과 설명

- 수업 notebook의 코드를 직접 옮겨 실행한다.
- 입력 → 처리 → 출력이 어떻게 이어지는지 설명한다.
- 각 변수와 함수가 왜 필요한지 본인의 표현으로 적는다.
- 결과 이미지와 similarity 값이 의미하는 바를 해석한다.

코드를 실행한 것보다, 왜 그렇게 동작하며 결과를 어디까지 신뢰할 수 있는지 설명하는 것이 레포트의 핵심이다.

레포트 출처 안내

Software의 동작은 공식 문서와 **source code**가 가장 직접적인 근거일 수도

사용할 수 있는 출처	사용 방법
논문·학술서적	algorithm의 배경, 검증 결과, 알려진 한계
공식 documentation	함수의 parameter, 반환값, 권장 사용법
공식 GitHub repository	실제 구현, README, source code, release note

- 공식문서 URL 링크 사용시 출처 예시:
 - Fingerprints—Scikit-fingerprints documentation. (n.d.). Retrieved September 6, 2026, from <https://scikit-fingerprints.readthedocs.io/latest/modules/fingerprints.html>
 - 생성형 AI의 답변 자체를 출처로 사용하시면 최하점입니다.
- 생성형 AI의 답변이 출처를 가져왔다면 그 출처를 기록하세요.
근데 그 출처가 매칭되지 않는 내용이라면 최하점입니다!
- 참고로 생화학·약학 도메인 지식에 의거하여 컴퓨팅 결과를 해석할 때는 생화학 논문, 약학 논문, 교과서를 주요 출처로 삼으세요.